

Modelado Fractal de Patologías Vasculares

Sosa, Diana Abril - 46.514.024; Merlo, Wilson - 44.826.264; Lugo, Renata Aylén - 44.572.954;
Romero, Francisco Ignacio - 45.529.404

FACENA UNNE, Teoría de la Computación, 4to Año, dianasosa1771@gmail.com

FACENA UNNE, Teoría de la Computación, 4to Año, julimerlo8@gmail.com

FACENA UNNE, Teoría de la Computación, 4to Año, lugorenata26@gmail.com

FACENA UNNE, Teoría de la Computación, 4to Año, fran92162@gmail.com

Resumen

Se diseñó e implementó un analizador léxico-sintáctico completo para un lenguaje formal inspirado en los Sistemas de Lindenmayer (L-Systems), orientado al modelado de redes vasculares. Se definió una Gramática Libre de Contexto (GLC) no ambigua expresada en notación BNF, con producciones que representan la estructura ramificada de vasos sanguíneos sanos y patológicos. Se construyó un lexer que tokeniza las cadenas de entrada y un parser recursivo descendente LL(1) que valida la estructura gramatical y construye el Árbol Sintáctico Abstracto (AST). El sistema diferencia tejido sano (símbolo V) de tejido cancerígeno (símbolo C) mediante la representación gráfica generada a partir del AST, con segmentos verdes para tejido normal y rojos para tejido patológico. Se validaron diez casos de aceptación y diez de rechazo, confirmando que el analizador reconoce correctamente cadenas válidas, detecta errores léxicos y sintácticos, y reporta línea y columna del error.

1. Introducción

Si bien históricamente los Sistemas de Lindenmayer han sido aplicados a la simulación botánica, el presente proyecto propone una transposición de este modelo matemático hacia el campo del diagnóstico médico asistido por computadora. Las redes vasculares y el árbol bronquial humano exhiben un comportamiento de crecimiento inherentemente fractal.

Aprovechando esta similitud topológica, este trabajo detalla el desarrollo de un Sistema Automatizado de Reconocimiento Topológico Vascular (SART-V). La premisa central consiste en utilizar un analizador sintáctico (parser) no solo como motor de renderizado, sino como un autómata de diagnóstico: al traducir imágenes médicas a secuencias de caracteres, el sistema evaluará si la estructura anatómica del paciente cumple con las reglas sintácticas de una red biológica "sana".

2. Pregunta Guía de Investigación

¿Es posible modelar e implementar un autómata basado en Sistemas de Lindenmayer que, al interpretar cadenas de texto que representan redes vasculares, permita detectar patrones morfológicos asociados a patologías mediante el análisis estructural de su visualización?

Esta pregunta sintetiza la hipótesis central del trabajo: que la gramática formal y los compiladores no son herramientas exclusivas del software, sino que pueden modelar y diagnosticar estructuras biológicas complejas cuando estas se representan como lenguajes formales.

3. Marco Teórico

3.1 Gramáticas Formales

Las gramáticas formales constituyen un modelo matemático utilizado para describir la estructura de un lenguaje mediante un conjunto de símbolos terminales, símbolos no terminales y reglas de

producción. Dentro de esta clasificación, las *Gramáticas Libres de Contexto (GLC)* permiten representar estructuras jerárquicas y recursivas, por lo que resultan adecuadas para el diseño de analizadores sintácticos. En este trabajo se empleó una GLC para definir formalmente la sintaxis del lenguaje utilizado en la representación de redes vasculares.

3.2 Analizador Léxico y Parser Descendente

El proceso de análisis sintáctico requiere una etapa previa de análisis léxico, en la cual la secuencia de caracteres de entrada es transformada en un conjunto de tokens pertenecientes al alfabeto del lenguaje. Durante esta etapa, los espacios en blanco, tabulaciones y saltos de línea son descartados al no poseer significado sintáctico.

Posteriormente, los tokens son procesados mediante un parser descendente recursivo, estrategia adecuada para gramáticas de tipo LL(1). En este enfoque, cada símbolo no terminal es implementado mediante una función recursiva encargada de verificar el cumplimiento de las reglas de producción y construir la estructura sintáctica correspondiente.

3.3 Geometría Fractal y Morfología Tumoral

Las redes vasculares y los procesos de angiogénesis presentan una organización ramificada con características fractales, donde las bifurcaciones mantienen patrones de autosimilitud a diferentes escalas. Este comportamiento puede representarse mediante los *Sistemas de Lindenmayer (L-Systems)*, los cuales describen el crecimiento biológico a través de reglas de producción aplicadas sobre un conjunto de símbolos.

En el presente trabajo, dicha representación fue adaptada mediante una Gramática Libre de Contexto, conservando los símbolos utilizados para describir el crecimiento de la estructura vascular. Los terminales V y C fueron utilizados para

representar tejido sano y tejido patológico, respectivamente.

3.4 El Parser como Herramienta de Validación

Desde la teoría de compiladores, un analizador sintáctico tiene como objetivo determinar si una cadena pertenece al lenguaje definido por una gramática. En este proyecto, dicho principio fue aplicado al modelado de redes vasculares, donde la cadena de entrada representa la estructura de un vaso sanguíneo y la gramática establece las configuraciones consideradas válidas.

En consecuencia, la aceptación de una cadena indica que la estructura respeta las reglas sintácticas definidas, mientras que su rechazo evidencia la existencia de una construcción inválida dentro del lenguaje especificado. Asimismo, durante el análisis se genera un Árbol Sintáctico Abstracto (AST), el cual representa de manera jerárquica la estructura reconocida por el parser.

4. Diseño de la Gramática

4.1 Definición formal en BNF

El alfabeto de la gramática define todos los símbolos válidos del sistema:

Tabla 1. Descripción de símbolos y acciones del alfabeto

Símbolo	Tipo	Acción	Significado
F	Constante gráfica	Avanza dibujando	Tejido formado
V	Variable sana	Marca tejido sano	Célula normal
C	Variable cancerígena	Marca patología	Célula cancerígena
+	Op. ángulo	Gira a la derecha	Bifurcación derecha
-	Op. ángulo	Gira a la izquierda	Bifurcación izquierda
[	Op. pila	PUSH posición	Inicio de rama
]	Op. pila	POP posición	Fin de rama

4.2 Conjunto de Producciones en Formato BNF

La gramática completa del sistema, expresada en notación Backus-Naur Form (BNF), es:

```

<vaso> ::= <tronco> "[" <rama> "]" "[" <rama> "]"
<tronco> ::= "F" "F" "F"
<rama> ::= <giro> "F" <tejido>
           | <giro> "F" <tejido> "[" <rama> "]" "[" <rama>
           "]"
<giro> ::= "+" | "-"
<tejido> ::= "V" | "C"

```

4.3 Explicación de Cada Producción

<vaso>: Regla axioma (raíz). Define que la topología vascular completa se compone de un tronco principal seguido por exactamente dos ramificaciones delimitadas por [y].

<tronco>: Tres constantes "F" consecutivas (FFF) que modelan la arteria principal del sistema, desde donde se originarán las ramificaciones secundarias.

<rama>: Producción recursiva unificada. Consume la dirección (<giro>), el segmento vascular ("F") y el estado biológico (<tejido>). Si detecta, asume bifurcación recursiva; caso contrario, procesa como hoja terminal.

<giro>: Orientación angular de la rama. Puede ser derecha (+) o izquierda (-).

<tejido>: Condición biológica del terminal. Tejido sano (V) o patológico/tumoral (C), determinando el color en la etapa de renderizado.

4.4 Palabra Aceptada

Aplicando la gramática definida, la siguiente cadena constituye una palabra aceptada válida:

FFF[+FV[+FV][-FV]][-FC[-FC]][+FC]

Esta cadena es aceptada porque todos sus símbolos pertenecen al alfabeto declarado, los corchetes están balanceados (bracket matching), y la estructura respeta el orden definido por las producciones BNF.

4.5 Derivación

La derivación completa desde el axioma <vaso> es:

```

<vaso>
→ <tronco> [<rama>][<rama>]
→ F F F [<rama>][<rama>]
→ F F F [<giro>F<tejido>
    [<rama>][<rama>]]
    [<giro>F<tejido>
    [<rama>][<rama>]]
→ FFF[+FV[+FV][-FV]]
   [-FC[-FC]][+FC]

```

4.6 Desglose de la Cadena

Cada segmento de la palabra aceptada tiene una interpretación directa:

Tabla 2. Desglose detallado de la cadena de derivación

Segmento	Producción	Interpretación
FFF	<tronco>	Tronco: 3 segmentos sanos base

[+FV[+FV][-FV]]	<rama>	Rama sana derecha con sub-bifurcaciones
[-FC[-FC][+FC]]	<rama>	Rama cancerígena (mutación C) y sub-bifurcaciones de angiogénesis
+	<giro>	Giro derecha (rama sana)
-	<giro>	Giro izquierda (rama cancerígena)
V	<tejido>	Terminal sano (verde)
C	<tejido>	Terminal patológico (rojo)

5. Implementación

5.1 Analizador Léxico

El analizador léxico se encarga de recibir el código fuente en formato de cadena de texto y convertirlo en una lista secuencial de tokens. Para optimizar el rendimiento de este proceso, el algoritmo recorre el texto carácter por carácter y clasifica cada símbolo basándose en el alfabeto del lenguaje. En lugar de evaluar múltiples expresiones regulares complejas, utiliza un mapeo directo contra un diccionario donde cada carácter válido se asocia a su token correspondiente.

```
SINGLE_CHAR_TOKENS = {
    "F": TokenType.F,
    "V": TokenType.V,
    "C": TokenType.C,
    "+": TokenType.PLUS,
    "-": TokenType.MINUS,
    "[": TokenType.LBRACKET,
    "]: TokenType.RBRACKET,
}

if char in SINGLE_CHAR_TOKENS:
    token = Token(
        SINGLE_CHAR_TOKEN
        S[char],
        char,
        self.line,
        self.column
    )
    tokens.append(token)
    self.advance()
    continue
```

Los caracteres de espacio en blanco, como saltos de línea y tabulaciones, son identificados y descartados de manera para no interferir con la lógica del analizador sintáctico, ya que de lo contrario tendría que hacerse un caso específico por cada palabra que pueda tener un espacio.

```
WHITESPACE = {" ", "\t", "\n", "\r"}
if char in WHITESPACE:
    self.advance()
    continue
```

Si detecta un carácter extraño que no coincida con ningún patrón del alfabeto, interrumpe el análisis lanzando un error léxico que registra la línea y la columna exacta de la anomalía

```
raise LexicalError(
    char, self.line,
    self.column
)
```

5.2 Analizador Sintáctico

Se eligió un analizador sintáctico descendente recursivo, implementado para gramáticas de tipo LL(1), este enfoque predictivo permite que cada símbolo no terminal se traduzca en una función específica como por ejemplo:

```
def parse_vaso(self) -> Vaso:
    tronco = self.parse_tronco()
    self.expect(TokenType.LBRACKET, "[" (inicio de la 1ra rama))
    rama_1 = self.parse_rama()
    self.expect(TokenType.RBRACKET, "]" (fin de la 1ra rama))

    self.expect(TokenType.LBRACKET, "[" (inicio de la 2da rama))
    rama_2 = self.parse_rama()
    self.expect(TokenType.RBRACKET, "]" (fin de la 2da rama))

    return Vaso(tronco=tronco, rama_1=rama_1, rama_2=rama_2)
```

Al mismo tiempo emplea la recursividad como mecanismo para procesar la jerarquía de niveles anidados. El motor opera de manera determinista mediante un lookahead de un token, lo que permite resolver derivaciones sin ambigüedad alguna. En la práctica, al analizar una rama, el parser consume la secuencia obligatoria y evalúa unívocamente si el símbolo siguiente corresponde a un corchete de apertura, determinando así si debe procesar sub-bifurcaciones recursivas o finalizar en una hoja terminal.

```
def parse_rama(self) -> Rama:
    <giro> F <tejido> (secuencia obligatoria) giro =
    self.parse_giro()
    self.expect(TokenType.F, "F" (segmento de la rama))
    tejido = self.parse_tejido()
```

```
primera_sub: Optional[Rama] = None
segunda_sub: Optional[Rama] = None
```

```
if self.check(TokenType.LBRACKET): self.advance()
```

```

primera_sub = self.parse_rama()
self.expect(TokenType.RBRACKET, "]" (fin de la 1ra
sub-rama)")

self.expect(TokenType.LBRACKET, "[" (inicio de la 2da
sub-rama)")
segunda_sub = self.parse_rama()
self.expect(TokenType.RBRACKET, "]" (fin de la 2da
sub-rama)")

return Rama(giro=giro, tejido=tejido,
primera_sub=primera_sub, segunda_sub=segunda_sub)

```

Para asegurar el control de excepciones y la robustez en la validación, el analizador sintáctico exige que cada símbolo coincida de manera exacta con el patrón esperado por la gramática. Cuando una función interna detecta la ausencia de un token obligatorio (como un corchete de cierre), en lugar de intentar recuperar el contexto descartando elementos, el bloque detiene el proceso y lanza la excepción `SyntaxErrorSARTV`. Esta interrupción captura el fallo y reporta con precisión la ubicación del problema, indicando qué token se esperaba y cuál se encontró en su lugar, evitando la generación de estados corruptos en el árbol de sintaxis abstracta. Por ejemplo en el caso de la entrada

FFF[+FX][-FV]

Se mostrará:

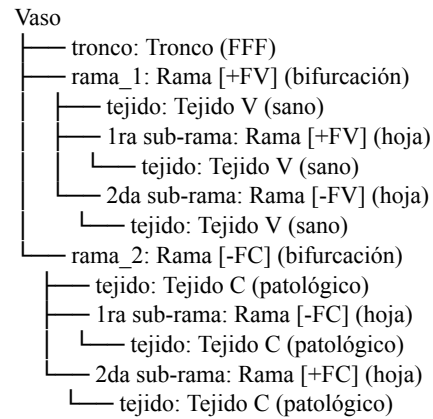
Se detectó una anomalía léxica en la línea 1, columna 7: el símbolo 'X' no constituye un terminal válido dentro del alfabeto Σ del sistema.

5.3 Representación Gráfica del AST

Al ingresar como entrada, por ejemplo, la secuencia:

FFF[+FV][-FC]

Su AST será:



5.4 Representación Gráfica

5.4.1 Estructura del Árbol Generado

La cadena FFF[+FV[+FV][-FV]][-FC[-FC][+FC]] se interpreta mediante el paradigma de gráficos de tortuga (Turtle Graphics). La tortuga lee la cadena de izquierda a derecha y ejecuta cada símbolo como instrucción de movimiento o ramificación. El árbol resultante se estructura en tres niveles:

Nivel 0 — tronco FFF (tejido base sano);

Nivel 1 — bifurcación principal, punto X;

Nivel 2 — sub-ramas con terminales V y C.

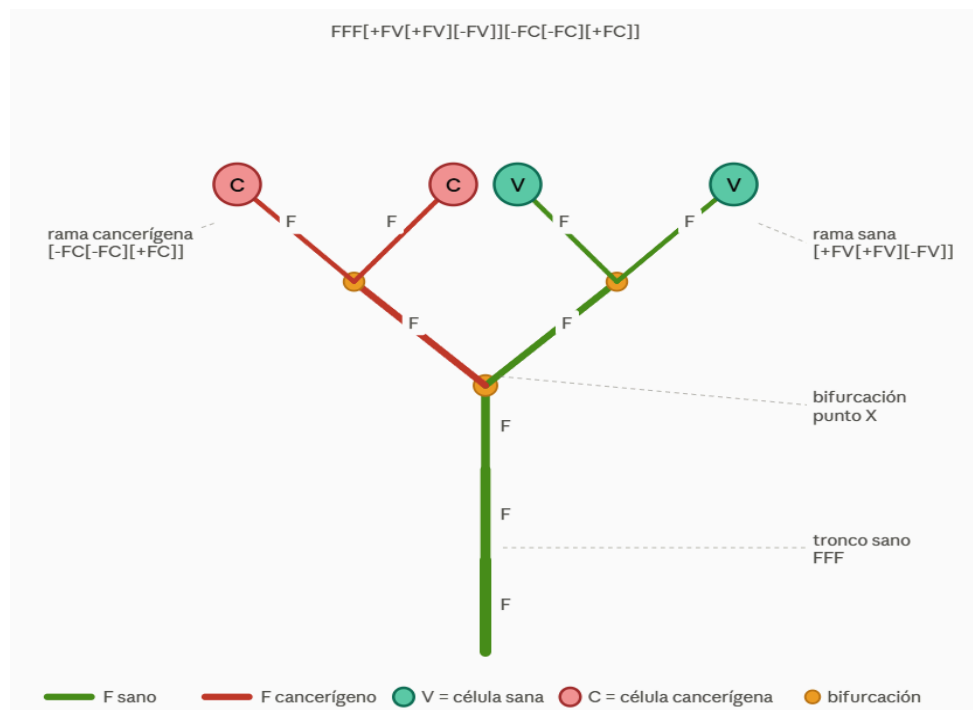


Fig. 1. Representación gráfica de FFF[+FV[+FV][-FV]][-FC[-FC][+FC]]

5.4.2 Interpretación Visual de los Símbolos

Rama sana [+FV][+FV][-FV]]: representada en color verde. Los segmentos F verdes y los terminales V reflejan crecimiento ordenado y simétrico, característico de proliferación celular controlada.

Rama cancerígena [-FC][-FC][+FC]]: representada en color rojo. Los segmentos F rojos y los terminales C reflejan crecimiento asimétrico y desordenado, característico de la angiogénesis tumoral.

5.4.3 Mecanismo Push/Pop en la Bifurcación

El mecanismo de pila permite al intérprete generar ramas sin perder el punto de origen. El flujo de ejecución para la cadena generada es:

F F F → Dibuja tronco. Pos. = punto X.
[→ PUSH: guarda punto X.
+FV → Rama sana principal.
[+FV] → Sub-rama sana derecha.
[-FV] → Sub-rama sana izquierda.
] → POP: vuelve al punto X.
[→ PUSH: guarda punto X.
-FC → Rama cancerígena principal.
[-FC] → Sub-rama cancerígena izq.
[+FC] → Sub-rama cancerígena der.
] → POP: vuelve al punto X.

5.5 Decisiones de Implementación

5.5.1 Simplificación mediante Bifurcaciones Binarias

Se adoptó una representación basada exclusivamente en bifurcaciones binarias: cada rama puede dividirse en exactamente dos ramas hijas. Esta decisión simplifica el diseño de la GLC y la implementación del parser. Al modelar la red vascular como un árbol binario recursivo se reduce la complejidad sintáctica, se facilita la construcción del AST y se mantiene la capacidad de representar estructuras fractales ramificadas. La restricción implica que cadenas con trifurcaciones o más serán rechazadas como sintácticamente inválidas, aunque sean posibles en la naturaleza.

5.5.2 Tratamiento de Espacios en Blanco

Los espacios en blanco, tabulaciones y saltos de línea no forman parte del alfabeto del lenguaje. El analizador léxico los ignora durante la tokenización, de manera que las siguientes representaciones son equivalentes para el sistema:

FFF[+FV][-FV]
FFF [+FV] [-FV]
FFF
[+FV]
[-FV]

6. Casos de Prueba y Resultados

Se diseñaron diez casos de aceptación y diez de rechazo para validar el funcionamiento del lexer y el parser.

6.1 Casos de prueba aceptados

Tabla 3. Casos de prueba de aceptación

I D	Cadena de entrada	Descripción	Resulta do Espera do	Resulta do Obteni do
V 0 1	FFF[+FV][-FV]	Vaso mínimo, dos ramas sanas (todo V).	Acepta	Acepta
V 0 2	FFF[+FV][-FC]	Una rama sana y una patológica.	Acepta	Acepta
V 0 3	FFF[+FC][-FC]	Ambas ramas patológicas (todo C).	Acepta	Acepta
V 0 4	FFF[-FV][+FC]	Giros invertidos (-, +).	Acepta	Acepta
V 0 5	FFF[+FV[+FV][-FV]][-FC][-FC][+FC]]	Bifurcación recursiva, profundidad 2.	Acepta	Acepta
V 0 6	FFF[+FV[+FV][-FC]][-FV]	Rama_1 con bifurcación mixta; rama_2 hoja.	Acepta	Acepta
V 0 7	FFF[+FC[+FC[+FC][-FC]][-FV]	Anidamiento profundo (profundidad 3).	Acepta	Acepta
V 0 8	FFF [+FV] [-FC]	Igual a V02 con espacios en blanco.	Acepta	Acepta (ignorando espacios en blanco)
V 0 9	FFF\n[+FV]\n[-FC]	Igual a V02 con saltos de línea.	Acepta	Acepta (ignorando saltos de línea)
V 1 0	FFF[+FC[+FV][-FV]][-FC[+FC][-FV]]	Anidamiento simétrico con tejidos mixtos.	Acepta	Acepta

6.2 Casos de prueba rechazados

Tabla 4. Casos de prueba de rechazo y errores sintácticos

I D	Cadena de entrada	Descripción	Resultado Esperado	Resultado Obtenido
I01	FF[+FV][-FV]	Tronco incompleto (solo FF).	Rechaza	Rechaza (error sintáctico)
I02	FFFF[+FV][-FV]	Tronco con un segmento de más.	Rechaza	Rechaza (error sintáctico)
I03	FFF[+FV]	Falta la segunda rama obligatoria.	Rechaza	Rechaza (error sintáctico)
I04	FFF[+FV][-FV][+FV]	Tercera rama de más a nivel raíz.	Rechaza	Rechaza (error sintáctico)
I05	FFF[+FV][-FC]	Corchete de cierre faltante.	Rechaza	Rechaza (error sintáctico)
I06	FFF[+FV][-FX]	Carácter 'X' no pertenece al alfabeto Σ .	Rechaza	Rechaza (error léxico)
I07	FFF[*FV][-FV]	Carácter '*' no pertenece a Σ .	Rechaza	Rechaza (error léxico)
I08	FFF[+FV][-FC]extra	Texto sobrante tras cadena válida.	Rechaza	Rechaza (error léxico)
I09	[+FV][-FV]	Falta el tronco FFF al inicio.	Rechaza	Rechaza (error sintáctico)
I10	FFF[+FV][-FC]	Corchete de cierre extra al final.	Rechaza	Rechaza (error sintáctico)

7. Discusión y Análisis

Durante el desarrollo del proyecto fue necesario adaptar los Sistemas de Lindenmayer al funcionamiento de un parser descendente recursivo. Debido a que los L-Systems realizan sustituciones en paralelo y un parser procesa la entrada de forma secuencial, se optó por representar el crecimiento vascular mediante una Gramática Libre de Contexto, conservando los símbolos característicos del modelo original.

Otra decisión importante fue implementar un parser descendente recursivo, ya que la naturaleza jerárquica de las bifurcaciones permitió representar cada rama mediante llamadas recursivas y construir el AST de manera sencilla. Además, los espacios en blanco se eliminaron durante el análisis léxico, manteniendo la gramática simple y compatible con un análisis LL(1).

Finalmente, se adoptó una representación basada en bifurcaciones binarias para reducir la complejidad del lenguaje y facilitar la implementación del analizador, manteniendo una estructura suficiente para representar el comportamiento fractal del sistema.

Como trabajo futuro, podría ampliarse el lenguaje para representar estructuras vasculares más complejas, incorporando bifurcaciones con más de dos ramas y nuevas características morfológicas. Asimismo, podría desarrollarse una interfaz gráfica que permita visualizar el proceso de análisis sintáctico y la construcción del Árbol Sintáctico Abstracto (AST) en tiempo real. Finalmente, el modelo podría extenderse para admitir una mayor variedad de patologías vasculares, manteniendo el enfoque basado en gramáticas formales y analizadores sintácticos.

8. Conclusión

El desarrollo de este proyecto deja en evidencia que las gramáticas formales y los analizadores sintácticos poseen un enorme potencial de aplicación fuera del desarrollo clásico de software. Al conceptualizar estructuras biológicas complejas como lenguajes formales, se demuestra que es posible utilizar la teoría de compiladores para representar y analizar estructuras anatómicas; una anomalía morfológica puede ser detectada y aislada como un patrón estructural atípico dentro de una gramática válida, sin que el flujo de ejecución colapse.

Se destaca la importancia de adaptar modelos matemáticos puros para viabilizar su procesamiento algorítmico. Si bien los Sistemas de Lindenmayer representan el crecimiento biológico mediante mutaciones simultáneas en paralelo, su implementación en un analizador tradicional requiere transponer su semántica a una Gramática Libre de Contexto secuencial y determinista.

Un parser descendente recursivo es la arquitectura idónea para abordar morfologías fractales, ya que su construcción desde la raíz hacia las hojas imita la jerarquía anatómica de un árbol vascular. La recursividad es la herramienta conceptual más eficiente para manejar bifurcaciones con profundidad desconocida.

A nivel arquitectónico, la separación de responsabilidades entre lexer, parser y módulo de visualización garantiza que el analizador sintáctico reciba datos puros y mantenga intacta su eficiencia y su propiedad LL(1).

9. Bibliografía

- [1] R. A. Lizárraga Morales, "Manual 2, Manual Técnico", Dept. de Ciencias de la Computación, Universidad Autónoma de San Luis Potosí, San Luis Potosí, México, 2006.
- [2] J. B. Ramírez, "Generación de contenido procedural: Un acercamiento a los Sistemas Lindenmayer", Reporte Técnico, Universidad de Guanajuato, Guanajuato, México, 2024.
- [3] R. Rincón Blanco, "Representación gráfica de paisajes generados mediante el uso de fractales", Trabajo de Fin de Grado, E.T.S. de Ingeniería de Sistemas Informáticos, Universidad Politécnica de Madrid, Madrid, España, 2022.
- [4] F. E. Castillo-Barrera, "Generación automática de música mediante gramáticas de sistemas L", Artículo de Investigación, Facultad de Ingeniería, Universidad Autónoma de San Luis Potosí, San Luis Potosí, México, 2006.